Inventory Management System for B2B SaaS

CASE STUDY SUBMISSION

Name : Ajinkya Gavhane
Role: Backend Engineer intern
MIT Academy of Engineering Alandi,Pune

Part 1: Code Review & Debugging

A previous intern wrote this API endpoint for adding new products. **Something is wrong** - the code compiles but doesn't work as expected in production.

```
@app.route('/api/products', methods=['POST'])
def create_product():
    data = request.json

    # Create new product
    product = Product(
        name=data['name'],
        sku=data['sku'],
        price=data['price'],
        warehouse_id=data['warehouse_id']
    )

    db.session.add(product)
    db.session.commit()

    # Update inventory count
    inventory = Inventory(
        product_id=product.id,
        warehouse_id=data['warehouse_id'],
        quantity=data['initial_quantity']
    )

    db.session.add(inventory)
    db.session.commit()

    return {"message": "Product created", "product_id": product.id}
```

Before jumping into the fixes, I want to be upfront about my approach, I've ordered these by production severity rather than the order they appear in the code. A bug that silently corrupts data is more dangerous than one that crashes loudly, and in a B2B inventory system where suppliers and stock levels are involved, silent failures are the ones that cause real business damage. I found 7 issues ,some are straightforward defensive coding gaps, a couple are business logic problems that would only show up under specific conditions in production.

Issue 1) No Transaction Management

The problem: The code does two separate db.session.commit() calls. If the product saves successfully but the inventory insert fails , you end up with a product that has no inventory record and no way to detect it without a manual audit query.

```
db.session.add(product)
db.session.commit()     # ← if this succeeds...

db.session.add(inventory)
db.session.commit()     # ← ...and this fails, data is now inconsistent
```

Production impact: In a B2B inventory system, a product that appears in search but has no stock record means staff could attempt to sell or reorder something the system can't track.

Fix: Single transaction, both records commit together or neither does.

```
try:
    db.session.add(product)
    db.session.flush()      # gets product.id without committing yet
    db.session.add(inventory)
    db.session.commit()     # one commit — atomic
except Exception:
    db.session.rollback()
    return {"error": "Internal server error"}, 500
```

Issue 2) No SKU Uniqueness Check

The problem: The requirement says SKUs must be unique, but there's no check before inserting. Relying only on a DB-level UNIQUE constraint means the error surfaces as an unhandled IntegrityError which becomes a raw 500 with a database error message exposed to the client.

Production impact: Duplicate SKUs break inventory tracking, analytics, and billing. Also leaks DB internals in error responses.

Fix: Explicit check with a proper 409 response.

```
if Product.query.filter_by(sku=sku).first():
    return {"error": "SKU already in use"}, 409
```

Issue 3) Race Condition on SKU (Concurrency)

The problem: Even with the check above, two simultaneous requests with the same SKU can both pass the filter_by check before either commits. This is a race condition.

Production impact: Under load or in distributed deployments, duplicate SKUs can still slip through.

Fix: The DB-level UNIQUE constraint acts as the final guard. Catch the IntegrityError explicitly and return a clean 409 don't let it become a 500.

```
from sqlalchemy.exc import IntegrityError

except IntegrityError:
    db.session.rollback()
    return {"error": "SKU already exists"}, 409
```

Issue 4) No Input Validation

The problem: data['name'], data['sku'] etc. are accessed directly. If any key is missing, Python raises a KeyError and Flask returns a 500. There's also no type validation price could arrive as a string, warehouse_id as null.

Production impact: API crashes on malformed requests instead of returning a useful 400. Makes debugging painful because logs show a traceback.

Fix:

```
data = request.json or {}

required = ['name', 'sku', 'price', 'warehouse_id']
missing = [f for f in required if not data.get(f)]
if missing:
    return {"error": f"Missing required fields: {missing}"}, 400
```

Issue 5) Price Stored Without Type Validation

The problem: price=data['price'] is passed directly with no casting or validation. If the frontend sends a string ("19.99"), behaviour depends on the ORM/DB column type and might store 0 or throw silently. Using float for money also introduces precision errors.

Production impact: Financial inaccuracies 19.999999 instead of 20.00. In a B2B SaaS handling supplier orders, this is a compliance risk.

Fix: Use Decimal, not float.

```
from decimal import Decimal

try:
    price = Decimal(str(data['price']))
    if price < 0:
        return {"error": "Price must be non-negative"}, 400
except:
    return {"error": "Invalid price format"}, 400
```

Issue 6) Wrong HTTP Status Code

The problem: Flask returns 200 by default. A successful resource creation should return 201 Created.

Production impact: Clients that check status codes can't distinguish "already existed" from "newly created".

Fix:

```
return {"message": "Product created", "product_id": product.id}, 201
```

Issue 7) No Authentication / Warehouse Ownership Check

The problem: There's no validation that the warehouse_id in the request belongs to the authenticated user's company. Any user could pass any warehouse_id.

Production impact: In a multi-tenant B2B system, this is a data isolation failure . One company's user could create products inside another company's warehouse.

Fix:

```
warehouse = Warehouse.query.filter_by(
    id=data['warehouse_id'],
    company_id=current_user.company_id
).first()
if not warehouse:
    return {"error": "Warehouse not found"}, 404
```

> **Note:** I'd confirm with the team whether this is already handled in auth middleware before adding it here just flagging it as a risk if it isn't.

Complete Fixed Version

```
from decimal import Decimal
from sqlalchemy.exc import IntegrityError

@app.route('/api/products', methods=['POST'])
@login_required
def create_product():
    data = request.json or {}
```

```
# 1. Validate required fields
required = ['name', 'sku', 'price', 'warehouse_id']
missing = [f for f in required if not data.get(f)]
if missing:
    return {"error": f"Missing required fields: {missing}"}, 400

# 2. Validate and cast price
try:
    price = Decimal(str(data['price']))
    assert price >= 0
except:
    return {"error": "Price must be a non-negative number"}, 400

# 3. Handle optional initial_quantity (defaults to 0)
try:
    quantity = int(data.get('initial_quantity', 0))
    assert quantity >= 0
except:
    return {"error": "Quantity must be a non-negative integer"}, 400

# 4. Warehouse ownership check (multi-tenant isolation)
warehouse = Warehouse.query.filter_by(
    id=data['warehouse_id'],
    company_id=current_user.company_id
).first()
if not warehouse:
    return {"error": "Warehouse not found"}, 404

# 5. SKU uniqueness check (+ IntegrityError as fallback for race condition)
if Product.query.filter_by(sku=data['sku']).first():
    return {"error": "SKU already in use"}, 409

# 6. Single atomic transaction
try:
    product = Product(
        name=data['name'],
```

```
        sku=data['sku'],
        price=price
    )
    db.session.add(product)
    db.session.flush()  # get product.id without committing

    inventory = Inventory(
        product_id=product.id,
        warehouse_id=data['warehouse_id'],
        quantity=quantity
    )
    db.session.add(inventory)
    db.session.commit()

except IntegrityError:
    db.session.rollback()
    return {"error": "SKU already exists"}, 409  # race condition fallback

except Exception:
    db.session.rollback()
    return {"error": "Internal server error"}, 500

return {"message": "Product created", "product_id": product.id}, 201
```

PART 2 : **Database Design**

I've designed the schema to cover what's stated, made reasonable assumptions where things were unclear, and listed the questions I'd bring to the product team before considering this finalized. I've used SQL DDL since it's unambiguous about types, constraints, and relationships.

Schema

---

companies

```
CREATE TABLE companies (
   id        SERIAL PRIMARY KEY,
   name      VARCHAR(255) NOT NULL,
   created_at TIMESTAMPTZ NOT NULL DEFAULT NOW()
);
```

warehouses

```
CREATE TABLE warehouses (
   id        SERIAL PRIMARY KEY,
   company_id INT NOT NULL REFERENCES companies(id),
   name      VARCHAR(255) NOT NULL,
   location  TEXT,
   created_at TIMESTAMPTZ NOT NULL DEFAULT NOW()
);

CREATE INDEX idx_warehouses_company ON warehouses(company_id);
```

suppliers

```
CREATE TABLE suppliers (
   id          SERIAL PRIMARY KEY,
   name        VARCHAR(255) NOT NULL,
   contact_email VARCHAR(255),
   created_at   TIMESTAMPTZ NOT NULL DEFAULT NOW()
);
```

products

```
CREATE TABLE products (
   id              SERIAL PRIMARY KEY,
   company_id        INT NOT NULL REFERENCES companies(id),
   name              VARCHAR(255) NOT NULL,
```

```
    sku               VARCHAR(100) NOT NULL,
    price             NUMERIC(12, 2) NOT NULL CHECK (price >= 0),
    product_type      VARCHAR(50) NOT NULL DEFAULT 'standard',
    low_stock_threshold INT NOT NULL DEFAULT 10,
    is_active         BOOLEAN NOT NULL DEFAULT TRUE,
    created_at        TIMESTAMPTZ NOT NULL DEFAULT NOW(),

    UNIQUE (company_id, sku)
    -- SKU unique per company, not globally (assumption — noted in gaps)
);

CREATE INDEX idx_products_company ON products(company_id);
```

product_suppliers *(many-to-many)*

```
CREATE TABLE product_suppliers (
    product_id  INT NOT NULL REFERENCES products(id),
    supplier_id INT NOT NULL REFERENCES suppliers(id),
    is_primary  BOOLEAN NOT NULL DEFAULT FALSE,
    PRIMARY KEY (product_id, supplier_id)
);
```

inventory

```
CREATE TABLE inventory (
    id          SERIAL PRIMARY KEY,
    product_id   INT NOT NULL REFERENCES products(id),
    warehouse_id INT NOT NULL REFERENCES warehouses(id),
    quantity     INT NOT NULL DEFAULT 0 CHECK (quantity >= 0),
    updated_at   TIMESTAMPTZ NOT NULL DEFAULT NOW(),

    UNIQUE (product_id, warehouse_id)
);

CREATE INDEX idx_inventory_product   ON inventory(product_id);
CREATE INDEX idx_inventory_warehouse ON inventory(warehouse_id);
```

inventory_logs

```
CREATE TABLE inventory_logs (
   id            SERIAL PRIMARY KEY,
   inventory_id   INT NOT NULL REFERENCES inventory(id),
   changed_by     INT REFERENCES users(id),
       change_type        VARCHAR(50)  NOT  NULL,  --  'sale',  'restock',
'adjustment', 'transfer'
   quantity_delta INT NOT NULL,
   quantity_after INT NOT NULL,
   note          TEXT,
   created_at     TIMESTAMPTZ NOT NULL DEFAULT NOW()
);

CREATE          INDEX          idx_inventory_logs_inventory          ON
inventory_logs(inventory_id, created_at DESC);
```

product_bundles

```
CREATE TABLE product_bundles (
   bundle_product_id    INT NOT NULL REFERENCES products(id),
   component_product_id INT NOT NULL REFERENCES products(id),
   quantity             INT NOT NULL DEFAULT 1,
   PRIMARY KEY (bundle_product_id, component_product_id)
);
```

users ***(referenced in inventory_logs for audit trail)***

```
CREATE TABLE users (
   id        SERIAL PRIMARY KEY,
   company_id INT NOT NULL REFERENCES companies(id),
   email      VARCHAR(255) NOT NULL UNIQUE,
   created_at TIMESTAMPTZ NOT NULL DEFAULT NOW()
);
```

Gaps , Questions I'd Ask the Product Team

Inventory & Sales

- What counts as "recent sales activity" 7 days, 30 days? Is this configurable per company or a global constant?
- Should product returns be logged in inventory_logs? If yes, does it reverse the sale or create a separate entry?
- If a product is transferred between warehouses, is that two log entries (one negative, one positive) or a single "transfer" record?

Bundles

- Can bundles contain other bundles (nested)? This changes the schema significantly .
- When a bundle is sold, does it decrement bundle-level inventory or component-level inventory? Right now I've stored both but the business rule isn't clear.

Suppliers

- Can a product have multiple suppliers? I've modelled it as many-to-many with an is_primary flag, but if it's always one supplier, a simple foreign key on products is simpler and faster.
- Are suppliers global across the platform or scoped per company**?**

Low Stock

- Is low_stock_threshold per product, per warehouse, or per product-type? I put it on the products table but if it varies per warehouse, it belongs on inventory instead.
- Can the threshold change over time, and do we need to track that history?

Multi-tenancy

- Is SKU unique globally or per company? I've assumed per company different businesses often reuse manufacturer SKUs. If it's global, the unique index changes.

Design Decisions

NUMERIC(12,2) instead of FLOAT for price Float is the wrong type for money NUMERIC stores exact decimal values.

UNIQUE(company_id, sku) instead of UNIQUE(sku) SKUs are unique within a company, not across the whole platform. A global unique constraint would prevent two different companies from using the same manufacturer SKU, which is an unnecessary restriction.

quantity_after stored in inventory_logs, not just delta Storing only the delta means you'd need to replay the entire log to reconstruct stock at a point in time. Storing quantity_after makes point-in-time auditing a single lookup.

Separate product_suppliers junction table A product can have multiple suppliers (backup supplier, regional supplier). Many-to-many with an is_primary flag handles this cleanly. If the team confirms it's always one supplier, I'd simplify this to a supplier_id on the products table.

product_bundles as a self-referential table Using a separate table rather than a JSON column keeps referential integrity intact and makes querying component relationships straightforward with a standard JOIN.

Descending index on inventory_logs(created_at) The most common query will be "what changed recently." A DESC index on created_at makes that fast without a full scan.

CHECK (quantity >= 0) on inventory Prevents negative stock at the DB level. Worth confirming with the team some businesses allow negative stock for pre-orders. If that's needed, this constraint comes off.

Part 3 Low Stock Alerts API

Assumptions

I'm stating these upfront so the team can correct them rather than discover them later in production:

- "Recent sales activity" = at least one sale in the last 30 days. I'd make this configurable but used 30 as a sensible default.
- Low stock = inventory.quantity < product.low_stock_threshold
- days_until_stockout = current quantity / average daily sales over last 30 days. If no sales data exists, this returns null — not 0, because 0 implies immediate stockout which would be misleading.
- For products with multiple suppliers, I'm returning the primary supplier (is_primary = TRUE). If none is marked primary, I return the first one. This needs a product team decision.
- Bundle stock alerting is on the bundle product itself, not its components. Component-level alerting for bundles needs a separate discussion.
- The endpoint is company-scoped and the authenticated user must belong to that company.

Implementation

```
from flask import jsonify, g

from sqlalchemy import func

from datetime import datetime, timedelta

from models import db, Company, Product, Inventory, Warehouse, Supplier, ProductSupplier, InventoryLog


RECENT_SALES_DAYS = 30  # would move to company-level config in production
```

```
@app.route('/api/companies/<int:company_id>/alerts/low-stock',
methods=['GET'])
@login_required
def low_stock_alerts(company_id):

    # Auth: user must belong to this company
    if g.current_user.company_id != company_id:
        return jsonify({"error": "Forbidden"}), 403

    company = Company.query.get(company_id)
    if not company:
        return jsonify({"error": "Company not found"}), 404

    recent_date = datetime.utcnow() - timedelta(days=RECENT_SALES_DAYS)

    # Step 1: Get all products for this company that are below threshold
    # One row per (product, warehouse) pair
    low_stock_rows = (
        db.session.query(Product, Inventory, Warehouse)
        .join(Inventory, Inventory.product_id == Product.id)
        .join(Warehouse, Warehouse.id == Inventory.warehouse_id)
        .filter(
            Warehouse.company_id == company_id,
```

```
        Product.is_active == True,
        Inventory.quantity < Product.low_stock_threshold
    )
    .all()
)

# Step 2: For each low-stock row, check for recent sales activity
# and calculate days until stockout
# Note: this is an N+1 query — acceptable for now, worth batching if alerts grow large
alerts = []

for product, inventory, warehouse in low_stock_rows:

        # Check recent sales in inventory_logs for this specific (product, warehouse)
    total_sold = (
        db.session.query(func.sum(func.abs(InventoryLog.quantity_delta)))
        .filter(
            InventoryLog.inventory_id == inventory.id,
            InventoryLog.change_type == 'sale',
            InventoryLog.created_at >= recent_date
        )
        .scalar() or 0
```

```
)

# Skip products with no recent sales — per business rule
if total_sold == 0:
    continue

avg_daily_sales = total_sold / RECENT_SALES_DAYS

# Avoid division by zero; return null if velocity is somehow 0
days_until_stockout = (
    round(inventory.quantity / avg_daily_sales)
    if avg_daily_sales > 0 else None
)

# Step 3: Get primary supplier for this product
# Falls back to first available supplier if none marked primary
supplier = (
    db.session.query(Supplier)
    .join(ProductSupplier, ProductSupplier.supplier_id == Supplier.id)
    .filter(ProductSupplier.product_id == product.id)
    .order_by(ProductSupplier.is_primary.desc())
    .first()
)
```

```
    alerts.append({
        "product_id": product.id,
        "product_name": product.name,
        "sku": product.sku,
        "warehouse_id": warehouse.id,
        "warehouse_name": warehouse.name,
        "current_stock": inventory.quantity,
        "threshold": product.low_stock_threshold,
        "days_until_stockout": days_until_stockout,
        "supplier": {
            "id": supplier.id,
            "name": supplier.name,
            "contact_email": supplier.contact_email
        } if supplier else None
    })

# Sort by urgency: closest to stockout first, unknowns (null) at the bottom
alerts.sort(key=lambda a: (
    a["days_until_stockout"] is None,
    a["days_until_stockout"]
))
```

```
    # TODO: add pagination before this goes to production

    # Large companies could have hundreds of alerts — unbounded response is a problem


    return jsonify({

        "alerts": alerts,

        "total_alerts": len(alerts)

    }), 200
```

Edge Cases Handled

- **No supplier on record** — returns supplier: null instead of crashing. Handled via the if supplier else None check.
- **No recent sales** — product is skipped entirely per the business rule. Not an error, just filtered out.
- **days_until_stockout returns null, not 0** — if avg daily sales rounds to zero, returning 0 would imply immediate stockout. null is the honest answer.
- **Multiple suppliers** — returns primary supplier, falls back to first. Needs product team confirmation on the right behaviour.
- **Forbidden company access** — 403 returned before any DB query runs.
- **Company not found** — explicit 404, not a silent empty response.

What I'd Improve with More Time

- **N+1 query** — the supplier and sales lookups inside the loop hit the DB once per alert row. With more time I'd batch both into subqueries upfront using GROUP BY and IN clauses. For companies with large product catalogs this would matter noticeably.
- **Pagination** — the response is unbounded right now. A ?page= and ?limit= parameter should be added before this goes to production.

- **Caching** — low stock alerts are likely polled frequently from a dashboard. A short cache (5–15 min via Redis) would reduce DB load significantly without meaningfully affecting freshness.
- **Bundle component alerts** — currently not handled. Whether a bundle being low means "alert on the bundle" or "alert on whichever component is causing the shortage" needs a product decision first.